Question:1

Fill your code here

Java (11)

```java
class GrowthLogger {
    private int day;
    private double height;

    public GrowthLogger(double initialHeight) {
        day = 0;
        height = initialHeight;
    }

    public class PlantGrowth {
        public double recordGrowth(double growth) {
            day++;
            height += growth;
            return height;
        }
    }

    public double[] startMonitoring(int days, double dailyGrowth) {
        PlantGrowth plantGrowth = new PlantGrowth();
        double[] growthLogs = new double[days];
        for (int i = 0; i < days; i++) {
            growthLogs[i] = plantGrowth.recordGrowth(dailyGrowth);
        }
        return growthLogs;
    }
}
```

Clear

Compile & Run

Submit Code

Question:2

Fill your code here

Java (1

```java
class DateValidator {
    public static String getDayType(String dateStr) {
        String[] parts = dateStr.split(" ");
        int day = Integer.parseInt(parts[0]);
        int month = Integer.parseInt(parts[1]);
        int year = Integer.parseInt(parts[2]);

        LocalDate date = LocalDate.of(year, month, day);
        int dayOfWeek = date.getDayOfWeek().getValue();

        if (dayOfWeek == 6 || dayOfWeek == 7) {
            return "Weekend";
        } else {
            return "Weekday";
        }
    }
}
```

Clear

Compile & Run

Submit Code

Question:3

Fill your code here

Java (11)

```java
class DaysCalculatorImpl {

    public static DaysCalculator createCalculator() {
        return new DaysCalculator() {
            public int calculateDays(int[] date1, int[] date2) {

                if (date1[0] < 2000 || date1[0] > 2024 ||
                    date2[0] < 2000 || date2[0] > 2024) {
            throw new IllegalArgumentException("Year must be between 2000 and 2024");
                }

                int days1 = date1[0] * 365 + date1[1] * 30 + date1[2];
                int days2 = date2[0] * 365 + date2[1] * 30 + date2[2];

                int smallerYear = Math.min(date1[0], date2[0]);
                int largerYear = Math.max(date1[0], date2[0]);

                for (int year = smallerYear; year < largerYear; year++) {
                    if (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)) {

                        if (date1[0] > date2[0]) {
                            days1++;
                        } else {
                            days2++;
                        }
                    }
                }

                return Math.abs(days2 - days1);
            }
        };
    }
}
```

Clear | Compile & Run | Submit Code | Prev | Next

Question:4

Fill your code here

Java (11)

```java
class PhoneNumberValidatorLogic {
    public static void validatePhoneNumber(String phoneNumber) {
        if (!phoneNumber.matches("\\d+")) {
throw new InputMismatchException("Input mismatch. Please enter a valid 10-digit phone number.");
        }
        if (phoneNumber.length() != 10) {
throw new IllegalArgumentException("Invalid phone number: Must be a 10-digit number.");
        }
    }
}

class PhoneNumberValidatorHandler {
    public void handleValidation() {
        Scanner scanner = new Scanner(System.in);
        try {
            String phoneNumber = scanner.nextLine();
            PhoneNumberValidatorLogic.validatePhoneNumber(phoneNumber);
            System.out.print("Valid");
        } catch (InputMismatchException e) {
            System.out.print("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.print("Error: " + e.getMessage());
        } finally {
            scanner.close();
        }
    }
}
```

Clear | Compile & Run | Submit Code | Prev | Next

Question:5

Fill your code here

Java

```java
class NumberProcessorLogic {
    public static class NegativeNumberException extends Exception {
        public NegativeNumberException(String message) {
            super(message);
        }
    }

    public static int processInput(int number) throws NegativeNumberException {
        if (number < 0) {
            throw new NegativeNumberException(number + " is negative");
        }
        return 2 * number;
    }
}
```

Clear

Compile & Run

Submit Code